INTERVIEW AND JUDGE PREPARATION

Trace3D Technical Viva Question Bank

Seventy-five detailed questions and defensible answers across vision, AI, backend, frontend, testing, and deployment

Answer pattern: define the concept, connect it to the exact code path, state the tradeoff, and never claim stronger evidence than the data supports.

Repository: github.com/carrie-proane/SIH-26
Audited branch: main / commit 3088287
Prepared: 03 September 2026 / Asia-Kolkata

Contents

Use this document as a technical reference during implementation, review, and viva preparation.

A. Product and scientific framing 5

1. What problem does Trace3D solve? . . . 5
2. Can the system recover a surface that was never visible? . . . 5
3. Why not call AI completion a reconstruction? . . . 5
4. What is the project's strongest engineering principle? . . . 5
5. What is the current maturity of the AI model? . . . 5

B. Photogrammetry and geometry 5

6. What is Structure-from-Motion? . . . 5
7. What is bundle adjustment? . . . 5
8. What is reprojection error? . . . 5
9. Why is triangulation angle important? . . . 5
10. Why use sequential matching for video? . . . 5
11. Why not use every video frame? . . . 6
12. What is the difference between sparse and dense reconstruction? . . . 6
13. Why use both COLMAP and OpenMVS? . . . 6
14. Why use PLY? . . . 6
15. What coordinate frame does COLMAP use? . . . 6
16. What is a 7-DoF similarity transform? . . . 6
17. Why ENU? . . . 6
18. Is drone GPS enough for accurate measurement? . . . 6
19. Why search for a telemetry time offset? . . . 6
20. What can make SfM fail? . . . 6

C. Preprocessing and telemetry 6

21. How is frame sharpness measured? . . . 6
22. How is exposure measured? . . . 7
23. How is redundancy measured? . . . 7
24. Why temporal distribution after scoring? . . . 7
25. Why keep candidate frames separate from selected frames? . . . 7
26. Which telemetry formats are supported? . . . 7
27. Why is altitude reference important? . . . 7
28. Why record warnings instead of silently cleaning data? . . . 7
29. Why use segmentation masks? . . . 7
30. Why are local segmentation weights required? . . . 7

D. AI surface completion 7

31. What is the recommended input representation? . . . 7
32. What is a TSDF? . . . 7
33. Why separate unknown from free space? . . . 8
34. Why use a generative model? . . . 8
35. Why conditional latent diffusion rather than full-resolution voxel diffusion? . . . 8
36. What conditions feed the model? . . . 8
37. How do you prevent the network from changing observed surfaces? . . . 8
38. How do you prevent geometry in known free space? . . . 8
39. How is uncertainty estimated? . . . 8
40. Why are three samples the default? . . . 8
41. What loss functions are appropriate? . . . 8
42. What datasets would you use? . . . 8
43. Why is ShapeNet alone insufficient? . . . 8
44. How do you create training partials? . . . 8
45. How do you avoid train/test leakage? . . . 9
46. Which metrics matter most? . . . 9
47. Why report hidden and observed metrics separately? . . . 9
48. Is NeRF or 3D Gaussian Splatting the completion model? . . . 9
49. What is Depth Anything used for? . . . 9
50. Why run the AI runtime as an external process? . . . 9
51. What does the external runtime receive and return? . . . 9
52. What happens if weights are missing? . . . 9

E. Backend and API **9**

53. Why FastAPI and Pydantic? . . . 9
54. How are raw inputs protected? . . . 9
55. How are JSON writes made safe? . . . 9
56. How is path traversal prevented? . . . 9
57. Why use a thread pool? . . . 10
58. Which endpoints form the normal workflow? . . . 10
59. Why does the viewer manifest return HTTP 409 before artifacts exist? . . . 10
60. How are API and frontend types kept aligned? . . . 10

F. Frontend and visualization **10**

61. Why Three.js? . . . 10
62. How does the viewer prevent arbitrary artifact loading? . . . 10
63. Why can only evidence mode be measurable? . . . 10
64. Why not derive confidence from point color? . . . 10
65. How does point measurement work? . . . 10
66. Why robust scene bounds? . . . 10
67. What does the texture validity contract do? . . . 10

68. Why React state screens instead of one giant component? . . . 10

G. Testing, performance, and roadmap 11

69. What does the verification gate include? . . . 11
70. What is not covered by unit tests? . . . 11
71. What is the main frontend performance warning? . . . 11
72. What is the main backend maintainability risk? . . . 11
73. How would you productionize run execution? . . . 11
74. What should be demonstrated to judges? . . . 11
75. What is the shortest honest project summary? . . . 11

A. Product and scientific framing

1. What problem does Trace3D solve?

It turns a controlled drone video plus synchronized telemetry into a traceable 3D reconstruction, quality report, source-frame evidence, and browser viewer. Its differentiator is not merely creating a point cloud; it separates observed geometry, visual-only dense output, and AI-predicted geometry so users know what can and cannot support measurement.

2. Can the system recover a surface that was never visible?

Not as physical fact. A generative model can infer a plausible surface using learned priors, but multiple shapes can explain the same visible images. The result must be labelled a hypothesis and carry uncertainty/provenance.

3. Why not call AI completion a reconstruction?

Reconstruction implies recovery constrained by observations. Completion adds geometry primarily from a learned prior. Calling both the same thing hides a fundamental difference in evidence.

4. What is the project's strongest engineering principle?

Fail honestly. A real run never falls back to synthetic geometry, missing tools produce actionable failure/blocker reports, and the frontend loads only artifacts explicitly declared by the backend.

5. What is the current maturity of the AI model?

There is no trained completion checkpoint in the repository. The observed reconstruction pipeline, runtime contract, configuration, artifact validation, reporting, and UI mode are implemented. Data collection, training, calibration, and quantitative validation remain R&D work.

B. Photogrammetry and geometry

6. What is Structure-from-Motion?

SfM jointly estimates camera intrinsics/extrinsics and sparse 3D points from feature correspondences across overlapping images. COLMAP detects features, matches them, geometrically verifies pairs, initializes a seed pair, triangulates points, and incrementally registers more cameras with bundle adjustment.

7. What is bundle adjustment?

It jointly optimizes camera parameters and 3D point positions to minimize reprojection error. It is a nonlinear least-squares problem and is central to achieving globally consistent camera/point geometry.

8. What is reprojection error?

For a reconstructed 3D point, project it into an image using the estimated camera model and measure the pixel distance to the detected 2D feature. Lower error generally indicates better consistency, but low error alone does not guarantee correct scale or complete geometry.

9. Why is triangulation angle important?

Two nearly collinear viewing rays intersect poorly; small localization error creates large depth error. A larger but reasonable baseline/angle improves depth conditioning. The point confidence module uses triangulation angle along with view support and reprojection error.

10. Why use sequential matching for video?

Adjacent video frames are temporally ordered and likely overlap. Sequential matching avoids the quadratic cost of exhaustive all-pairs matching while preserving likely correspondences. The overlap window must be large enough for dropped/poor frames and actual motion.

11. Why not use every video frame?

Adjacent frames can be nearly identical, increasing compute without useful baseline. Blurry, overexposed, or redundant frames can also hurt matching. The pipeline selects quality-gated, temporally distributed keyframes.

12. What is the difference between sparse and dense reconstruction?

Sparse SfM estimates cameras and selected feature points. Dense MVS estimates depth/normal maps for many pixels and fuses them into a much denser cloud/mesh. Dense MVS still needs overlapping observed views and does not solve genuinely hidden surfaces.

13. Why use both COLMAP and OpenMVS?

COLMAP provides robust SfM and can provide dense PatchMatch when supported by the machine. OpenMVS is an external dense fallback for densification, meshing, refinement, and texturing, including the local Metal-aware setup in this repository.

14. Why use PLY?

PLY is straightforward for point clouds and meshes, supports coordinates/color and can be read by Three.js. It is not ideal for every production use case, but it is transparent and easy to validate.

15. What coordinate frame does COLMAP use?

COLMAP recovers an arbitrary similarity frame for monocular imagery: translation/orientation and especially scale are not tied to metres. The pipeline estimates a similarity transform from camera centres to telemetry-derived local ENU positions.

16. What is a 7-DoF similarity transform?

Three rotation parameters, three translation parameters, and one uniform scale. The Umeyama method estimates these by centering point sets, applying SVD to cross-covariance, resolving reflection, and computing scale/translation.

17. Why ENU?

East-North-Up is an intuitive local metric frame. WGS84 latitude/longitude/height are converted to ECEF and rotated around a local origin into metres, avoiding calculations directly in degrees.

18. Is drone GPS enough for accurate measurement?

No. Consumer GNSS can have metre-level error, altitude bias, drift, and timestamp mismatch. It is a soft scale/alignment prior. Survey-grade claims require control points, RTK/PPK, calibrated sensors, and independent validation.

19. Why search for a telemetry time offset?

Video timestamps and flight logs may not share the exact zero time. A shift can make correct camera trajectories appear spatially wrong. The pipeline searches a bounded offset and selects the robust alignment with lowest RMSE unless a manual/calibrated offset is supplied.

20. What can make SfM fail?

Insufficient overlap/parallax, blur, rolling shutter, repetitive or textureless surfaces, changing exposure, specular/transparent materials, moving objects, sky dominance, incorrect intrinsics, or a trajectory that only rotates from one position.

C. Preprocessing and telemetry

21. How is frame sharpness measured?

Using the variance of the Laplacian response. Edges create high-frequency response; a blurred image usually has lower variance. The value is scene-dependent, so the selector combines an absolute floor with a relative sharpness floor.

22. How is exposure measured?

The score penalizes pixels concentrated near black or white and rewards a useful intensity distribution. It is a heuristic quality signal, not radiometric calibration.

23. How is redundancy measured?

The scorer compares nearby frames using a structural-similarity-style measure. High similarity means low uniqueness, so the selector can avoid spending compute on nearly identical frames.

24. Why temporal distribution after scoring?

Selecting only the top global scores can cluster frames in one easy segment, leaving trajectory gaps. Temporal distribution preserves coverage while quality gates prevent low-quality frames from being added solely to fill quotas.

25. Why keep candidate frames separate from selected frames?

It makes decisions auditable and ensures COLMAP sees only the declared selection. Candidates remain available for contact-sheet review and operator overrides.

26. Which telemetry formats are supported?

Multiple DJI SRT dialects and CSV layouts with common header/unit variants. Both normalize into timestamp, latitude, longitude, altitude, altitude source, fix quality, and source row.

27. Why is altitude reference important?

Relative-to-launch, ellipsoidal, and mean-sea-level heights are different. Mixing them introduces a vertical bias. The normalized schema records `alt_source` rather than pretending all altitude values are equivalent.

28. Why record warnings instead of silently cleaning data?

Automatic repair can create plausible but wrong data. Structured warnings preserve provenance and let quality reports/UI communicate duration mismatch, gaps, bad fixes, or implausible speeds.

29. Why use segmentation masks?

Dynamic objects, sky, and irrelevant background can create unstable features and dense artifacts. Masks exclude those pixels. In primary-subject mode, a complete valid mask set is required because the intended geometry would otherwise be undefined.

30. Why are local segmentation weights required?

Reproducibility, privacy, offline operation, licensing, and version control. Silent downloads can change results or fail during a demo. AUTO mode falls back honestly; REQUIRED mode stops.

D. AI surface completion

31. What is the recommended input representation?

A sparse TSDF/SDF or occupancy volume with separate channels for observed surface, observation weight, known free space, unknown mask, geometric confidence, and optional semantic/image features.

32. What is a TSDF?

A truncated signed distance field stores the signed distance to the nearest surface within a band. Multiple depth observations can be fused by weighted averaging, and the zero level set becomes the surface extracted by Marching Cubes.

33. Why separate unknown from free space?

Known free space has been traversed by a camera ray before its measured surface and should not contain geometry. Unknown space was never constrained. Combining them teaches the model to erase hidden objects or to create surfaces in space known to be empty.

34. Why use a generative model?

Hidden geometry is multi-modal. A chair, statue, or facade can have multiple plausible backsides. A conditional diffusion model can sample alternatives, whereas deterministic regression often averages them into an implausibly smooth shape.

35. Why conditional latent diffusion rather than full-resolution voxel diffusion?

Latent diffusion reduces memory/compute while retaining stochastic generation. A decoder reconstructs an SDF from the latent; coarse-to-fine refinement recovers surface detail. Full dense 3D diffusion is expensive and scales poorly to outdoor scenes.

36. What conditions feed the model?

Partial TSDF/occupancy, observation/unknown/free-space masks, confidence, camera visibility, and optionally back-projected image features or semantic labels. Geometry and visibility should dominate; text/image priors should never override high-confidence evidence.

37. How do you prevent the network from changing observed surfaces?

Use an observed-region preservation loss, high training weight, and inference-time clamping of SDF values in high-confidence observed cells. Then apply a narrow transition band to join generated and observed fields.

38. How do you prevent geometry in known free space?

Ray-carve free space from every camera/depth map, supply it as an input channel, penalize predicted occupancy there, and validate each hypothesis by re-rendering depth/silhouette to source cameras.

39. How is uncertainty estimated?

Combine disagreement across stochastic hypotheses with geometric support/confidence. Calibrate the score on held-out partial/complete pairs and evaluate whether high uncertainty predicts high error.

40. Why are three samples the default?

One cannot show ambiguity; many are expensive. Three is a practical starting point for disagreement and demo visualization, not a statistically proven optimum. Tune after runtime/error studies.

41. What loss functions are appropriate?

Robust SDF L1 near the surface, balanced occupancy BCE, observed preservation, free-space violation, normal consistency, Eikonal regularization, multi-view rendered depth/silhouette loss, and the diffusion noise-prediction objective.

42. What datasets would you use?

Synthetic drone-like partial/complete pairs for scale; DTU/Tanks and Temples for reconstruction evaluation; ScanNet/ScanNet++ for scene priors; ShapeNet only for category/object work; and, most importantly, project-domain full reference scans for fine-tuning/testing.

43. Why is ShapeNet alone insufficient?

It is mostly clean canonical CAD objects, unlike large outdoor scenes with sensor noise, vegetation, windows, weather, occluders, and arbitrary scale. A model may learn category shapes but fail badly on drone reconstructions.

44. How do you create training partials?

Render depth from actual-like camera trajectories over a complete mesh, fuse only visible depths, ray-carve free space, mark the rest unknown, and inject measured pose/depth/mask noise. The complete mesh/SDF remains the target.

45. How do you avoid train/test leakage?

Split by physical object/site before rendering views. Adjacent frames or different partial paths over the same mesh cannot be divided across training and test.

46. Which metrics matter most?

Hidden-region F-score/precision/recall at metric thresholds, Chamfer distance, normal consistency, IoU, observed-region drift, free-space violations, boundary cracks, and uncertainty calibration.

47. Why report hidden and observed metrics separately?

Copying the observed 80 percent can dominate an overall score and hide total failure on the occluded 20 percent, which is the actual task.

48. Is NeRF or 3D Gaussian Splatting the completion model?

Not by itself. They optimize/render radiance and can synthesize nearby views, but visual quality does not prove watertight or metrically correct hidden geometry. They can supply features/appearance or a separate photoreal mode.

49. What is Depth Anything used for?

As an optional monocular depth prior, especially where MVS is weak. Its depth must be aligned to geometric scale and checked across views. It cannot make an unobserved backside measurable.

50. Why run the AI runtime as an external process?

It isolates heavy GPU dependencies, prevents FastAPI environment conflicts, supports independent packaging, permits resource/time limits, and gives a versioned request/output boundary that tests can fake deterministically.

51. What does the external runtime receive and return?

It receives `completion_request.json` containing observed geometry, camera poses, selected frames, coordinate frame, model path, sample count, and scientific rules. It returns a PLY mesh and uncertainty JSON. The backend validates, reports, registers, and serves them.

52. What happens if weights are missing?

The completion stage records `BLOCKED` and a structured warning. The observed sparse/dense run remains valid and can complete. No fake mesh is generated.

E. Backend and API

53. Why FastAPI and Pydantic?

FastAPI provides typed ASGI endpoints and automatic schema generation; Pydantic provides strict runtime validation, field bounds, literals/enums, and serialization for project/run contracts.

54. How are raw inputs protected?

`ProjectStore` copies each input into a project directory, records original name, size, media type, SHA-256 and provenance, and does not modify it. Run outputs live in separate run directories.

55. How are JSON writes made safe?

`atomic_json` writes to a temporary sibling and replaces the destination, reducing the chance of a partially written manifest if the process stops during serialization.

56. How is path traversal prevented?

IDs match a strict safe pattern, artifact access resolves only relative paths already present in the run's declared artifact list, and absolute/parent-traversal requests are rejected.

57. Why use a thread pool?

Run creation should return quickly while CPU/external tool work continues. A small bounded pool allows local concurrency. A production multi-machine system should replace it with a durable job queue.

58. Which endpoints form the normal workflow?

`POST /api/projects`, `POST /api/projects/{id}/runs`, repeated `GET /api/runs/{id}`, then `GET /api/runs/{id}/viewer-manifest` and declared artifact URLs. `/health` supports readiness.

59. Why does the viewer manifest return HTTP 409 before artifacts exist?

The run exists, but the viewer's required artifact set is not ready. `409 Conflict` distinguishes that state from a missing run (`404`) and prevents the UI from fabricating a partial success view.

60. How are API and frontend types kept aligned?

Pydantic models are authoritative on the backend; `domain/contracts.ts` mirrors the JSON. Integration and component tests exercise the exact payload. A future improvement is OpenAPI-based TypeScript generation to reduce manual drift.

F. Frontend and visualization

61. Why Three.js?

It provides WebGL scene/rendering abstractions, PLY/GLTF loaders, orbit controls, raycasting, materials, and geometry math while integrating cleanly with React-managed UI state.

62. How does the viewer prevent arbitrary artifact loading?

`declaredVisualArtifactUrls` builds a URL set from the backend manifest. The streamed loader refuses any URL not in that set, even if a caller provides one.

63. Why can only evidence mode be measurable?

Dense/textured models are presentation artifacts in the current contract and AI completion is inferred. `visualModeMeasurementEligible` requires Evidence mode and a backend eligibility flag.

64. Why not derive confidence from point color?

PLY RGB is photographic appearance. Red may be a red wall, not low confidence. Confidence is a separate validated JSON artifact in exact PLY vertex order.

65. How does point measurement work?

Three.js raycasting selects two visible 3D points, markers are added, and Euclidean distance is computed in the model coordinate frame. It is meaningful in metres only after valid local ENU alignment and evidence eligibility.

66. Why robust scene bounds?

A few extreme outliers can expand a normal bounding box so far that the main cloud becomes a dot. Quantile-based bounds reduce outlier influence before fitting the camera.

67. What does the texture validity contract do?

OpenMVS can encode unsupported atlas regions with a known empty color. The backend reports an accepted contract and the frontend filters triangles sampling unsupported regions rather than displaying obvious texture garbage.

68. Why React state screens instead of one giant component?

Setup, processing, and workspace have distinct state and responsibilities. Feature folders keep UI logic discoverable while `App.tsx` coordinates transitions and async work.

G. Testing, performance, and roadmap

69. What does the verification gate include?

Backend Pytest, Ruff, frontend Vitest, TypeScript/Vite production build, and Playwright browser test. External COLMAP/OpenMVS/FFmpeg and real-data quality still require environment/fixture checks.

70. What is not covered by unit tests?

Scientific performance on a representative real captured dataset, GPU compatibility across machines, large-file load/performance, complete browser visual QA on all artifacts, and trained model quality.

71. What is the main frontend performance warning?

The Three.js production chunk is about 600 kB before gzip and exceeds the configured warning limit. Lazy-loading the viewer route or Three.js loaders would reduce initial launch cost.

72. What is the main backend maintainability risk?

`pipeline.py` and `dense.py` are large. The new domain layout makes ownership clearer, but future work should split orchestration stages and OpenMVS/COLMAP providers into smaller modules with stage-level contracts.

73. How would you productionize run execution?

Use a durable database and object storage, a job queue with worker heartbeats/retries, idempotent stage artifacts, signed artifact access, authentication/authorization, resource quotas, structured logs, metrics/traces, and GPU worker isolation.

74. What should be demonstrated to judges?

Show input provenance, selected frames/contact sheet, sparse evidence, alignment/known-distance report, dense visual model, then an AI-predicted layer in a different color/mode with measurement disabled. Also show a deliberate missing-weights failure to prove the system does not fake success.

75. What is the shortest honest project summary?

"We reconstruct what the video observed using photogrammetry, align it into a local metric frame, and optionally generate plausible hidden surfaces with a learned prior. Observed and predicted geometry remain separate, and only validated observed evidence is measurable."